

Programming Fundamentals (CSP2151)

Assignment 2:	Individual programming assignment (C Program)
Assignment Marks:	Marked out of 50, worth 50% of unit
Due Date:	Check Canvas

Background Information

This assignment tests your understanding of and ability to apply the programming concepts we have covered in the unit so far, including the usage of variables, input and output, data types, selection, iteration, functions and data structures in C. Above all else, it tests *your ability to design and then implement a solution to a problem* using these concepts.

See the Program Requirements section for further details.

Academic Integrity and Misconduct

The **entirety of your assignment must be your own work** (unless otherwise referenced) and produced for the current instance of the unit. Any use of Generative AI e.g., ChatGPT, Gemini etc., and unreferenced content you did not create constitutes plagiarism, and is deemed an act of academic misconduct. All assignments will be submitted to plagiarism checking software which includes previous copies of the assignment, and the work submitted by all other students in the unit.

Remember that this is an **individual** assignment. *Never give anyone any part of your assignment – even after the due date or after results have been released. Do not work together with other students on individual assignments – you can help someone by explaining a concept or directing them to the relevant resources, but doing any part of the assignment for them or alongside them, or showing them your work, is inappropriate.* An unacceptable level of cooperation between students on an assignment is collusion, and is deemed an act of academic misconduct. If you are uncertain about plagiarism, collusion or referencing, simply contact your tutor, lecturer or unit coordinator.

You may be asked to **explain and demonstrate your understanding** of the work you have submitted. Your submission should accurately reflect your understanding and ability to apply the unit content.

Assignment Overview

You are required to design and implement a “Maths Test” program that generates and presents simple maths questions to the user for them to answer. Once the test has been completed, the program displays the user’s overall score/result.

The entirety of this program can be implemented in under 125 lines of code (although implementing additions may result in a longer program). *This number is not a limit or a goal.*

Program Output Example

To help you visualise the program, here is an example screenshot of the program being run:

```
Welcome to Maths Tester Pro.
Select a difficulty:
1) Easy
2) Medium
3) Hard
> easy
Invalid choice! Enter 1, 2 or 3.
> 1
Easy mode selected!

Question 1 of 5. You have 3 lives remaining.
What is 5 - 3? 2
Correct!

Question 2 of 5. You have 3 lives remaining.
What is 5 + 5? 10
Correct!

Question 3 of 5. You have 3 lives remaining.
What is 8 + 8? 19
Incorrect! The correct answer was 16.

Question 4 of 5. You have 2 lives remaining.
What is 1 + 2? 4
Incorrect! The correct answer was 3.

Question 5 of 5. You have 1 life remaining.
Challenge question!
What is 11 - 20? -9
Correct!

Test complete.
You scored 3/5 (60%).
Your grade is a Credit!
```

The program welcomes the user and then prompts them to select a difficulty. The user first tried entering “easy” and was told it was an invalid choice. They enter “1” and the program confirms that they have selected Easy mode. *The difficulty determines the number of questions, largest number used when generating a question, and number of questions that the user can get wrong.*

The program then generated 5 random questions (the last one being a challenge question with larger numbers) and presented them to the user, who answered 3 out of 5 of them correctly.

Finally, the program displayed the overall score and percentage, and the user’s grade (a Credit).

Pseudocode

As emphasised by the case study of Module 5, it is important to take the time to properly *design* a solution before starting to write code. Hence, this assignment requires you to *write and submit pseudocode of your program design* as well as the code for the program.

You will gain a lot more benefit from pseudocode if you actually attempt it *before* trying to code your program – even if you just start with a rough draft to establish the overall program structure, and then revise and refine it as you work on the code. This back and forth cycle of designing and coding is completely normal and expected, particularly when you are new to programming. *The requirements detailed on the following pages should give you a good idea of the structure of the program, allowing you to make a start on designing your solution in pseudocode.*

See Reading 3.3 for tips and advice regarding pseudocode.

Write a *pseudocode for each function* you define in your program. Ensure that the pseudocode for each of your functions clearly describes the parameters that the function receives and what the function returns back to the program.

Your solution must be modular and therefore **more than two functions**, including `ask_question` (detailed later in the assignment brief), are required in this assignment.

Programming Tip: Do not attempt to implement the entire program at once. Work on one small part (a single requirement or even just *part* of a requirement) and only continue to the next part once you have made sure that the previous part is working as intended *and that you understand it*.

It can also be useful to create separate little programs to work on or test small sections of complex code, allowing you to focus on just that part without the rest of the program getting in the way.

Program Requirements (Implement these requirements in “main.c” file)

In the following information, numbered points describe a *core requirement* of the program, and bullet points (in italics) are *additional details, notes and hints* regarding the requirement. Ask your tutor if you do not understand the requirements or would like further information.

1. Print a welcome message, and then prompt the user to select a difficulty by entering 1, 2 or 3.
2. Use a loop to re-prompt the user until a valid response (1, 2 or 3) is entered. Once a difficulty has been selected, print a message confirming the selected difficulty and set variables as follows:
 - *If “Easy” (1) was chosen... lives = 3, max_num = 10 and questions = 5*
 - *If “Medium” (2) was chosen... lives = 2, max_num = 25 and questions = 10*
 - *If “Hard” (3) was chosen... lives = 1, max_num = 50 and questions = 15*
 - *“lives” represents how many incorrect answers are permitted, “max_num” represents the largest number used when generating a question, and “questions” represents the number of questions.*
3. Set a “score” variable to 0, and then enter a loop that repeats `questions` times.
 - *“score” will be used to keep track of how many questions the user has answered correctly.*

The body of this loop must...

- 3.1. Print which question the user is up to out of the total number of `questions`, as well as how many lives that have remaining, e.g. “Question 1 of 5. You have 2 lives remaining.”
 - *Print “1 life remaining” rather than “1 lives remaining” if lives is 1.*
- 3.2. If the current question is *not* the final question of the test, use the “ask_question” function (detailed below) to generate and administer a question involving numbers between 1 and `max_num`.

If the current question *is* the final question of the test, print “Challenge question!” and use the “ask_question” function (user defined) to generate and administer a question involving numbers between `max_num` and `max_num` multiplied by 2.

- *e.g. The challenge question on Easy difficulty would use numbers between 10 and 20*
 - If the “ask_question” function returns a value of True, add one to the `score` variable. Otherwise, subtract 1 from the `lives` variable. If the `lives` variable is now 0, print “Out of lives, game over!” and immediately end the test (proceeding to Requirement 4).
4. Print a “test complete” message, followed by a message that displays the user’s `score` out of `questions`, and what percentage that represents, e.g. “You scored 3/5 (60%).”
 - *Round the percentage value to the nearest whole number.*

5. Print the user's grade, based upon their percentage. The grades are High Distinction (100-80%), Distinction (79-70%), Credit (69-60%), Pass (59-50%) and Fail (49-0%).

The “ask_question” Function

There are two points in Requirement 3.2 where the program must generate and administer a question. This is a self-contained task consisting of a number of steps, with the only difference being the minimum and maximum numbers to use. As such, it is ideal to create a function for this task.

You must create a function named “ask_question” that *receives two parameters*:

- “minimum”, an integer representing the smallest number to use in the question
- “maximum”, an integer representing the largest number to use in the question

The function should generate two random integers between `minimum` and `maximum`, and then randomly select a mathematical operator of either '+' or '-'. It should use these values to display a question, e.g. “What is 4 + 5?”, and prompt the user for their answer.

If the user answers correctly, the function should print “Correct!” and return the boolean value of True. Otherwise, the function should print “Incorrect!” and the correct answer, and return False.

The code that you design and write to implement this function is up to you, but you may find it useful to use the following C function:

- The “rand()” function, to generate random numbers within a range.

The definition of the function should be at the start of the program, and it should be called where needed in the program. Revise Module 4 and 10 if you are uncertain about defining and using functions, and be sure to implement it so that it receives and returns values exactly as described above.

Ensure that the function does exactly what is specified above and nothing more - it is important to adhere to the stated specifications of a function.

Additions and Enhancements (Implement these requirements in “main_additions.c” file)

Below are some minor additions and enhancements that you can make to the program to further test and demonstrate your programming ability.

- (2 marks) When prompting the user to choose a difficulty, make it so that the program accepts “1”, “e” or “easy” for Easy, “2”, “m” or “medium” for Medium, and “3”, “h” or “hard” for hard. A good approach is to convert the user’s input to lowercase, and use an “in” comparison to compare the user’s input to a tuple of options.
- (1 mark) Put everything after the creation of the `ask_question` function into a loop so that the test can be run repeatedly without having to re-run the program. At the end of the test, ask the user whether they wish to start again and break out of the loop if they don’t.
- (2 marks) Use arrays in your code.

Submission of Deliverables

Once your assignment is complete, submit both your **pseudocode** (in **PDF format**) and **source code** (“**main.c**” and “**main_additions.c**” files) to the appropriate location in the Assessments area of Canvas. *Zippping the files is not required.* An assignment cover sheet is not required, but be sure to **include your name and student number at the top of both files (not just in the filenames)**.

Marking Key

Criteria	Marks
Pseudocode These marks are awarded for submitting pseudocode which suitably represents the design of your source code. Pseudocode will be assessed on the basis of whether it clearly describes the steps of the program in English, and whether the program is well structured.	10
Functionality These marks are awarded for submitting source code that implements the requirements specified in this brief, in C. Code which is not functional or contains syntax errors will lose marks, as will failing to implement requirements as specified.	25
Additions and Enhancements These marks are awarded for correctly implementing all the additions and enhancements on page 6. Code which does not include these additions and enhancements will lose marks.	5
Code Quality These marks are awarded for submitting well-written source code that is well-formatted and demonstrates a solid understanding of the concepts involved. This includes appropriate use of commenting and adhering to best practise (e.g., modular programming).	10
Total:	50